

Database Management Systems

CPS 3740

Lecture 06

Computer Science, Kean University

Dr. Ching-yu (Austin) Huang

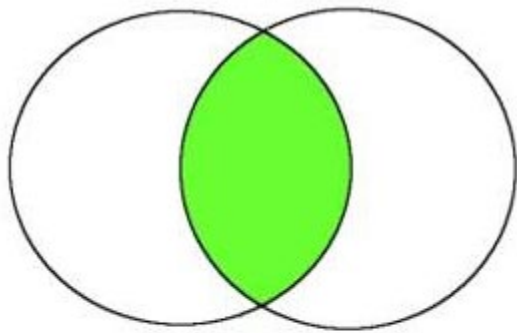
Instructor Paolien Wang

Outline

- SET, INTERSCT, UNION, SET DIFFERENCE
- DML – Subquery, ANY, ALL, IN
- Homework
- Project status

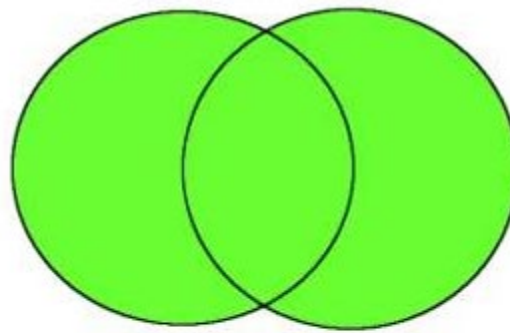
SET

- Intersect, Union, Minus
- MySQL doesn't support **INTERSECT**
- MySQL doesn't support **MINUS, EXCEPT**



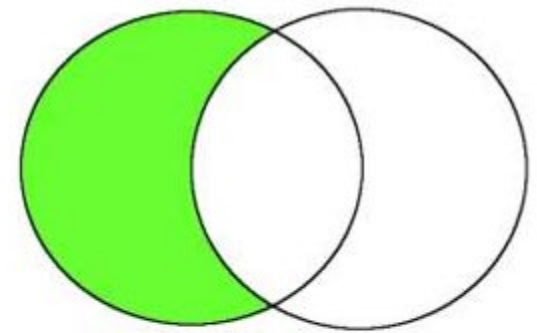
Set 1 **Set 2**

Intersect



Set 1 **Set 2**

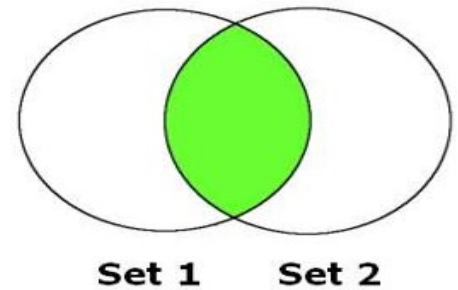
Union
Union All



Set 1 **Set 2**

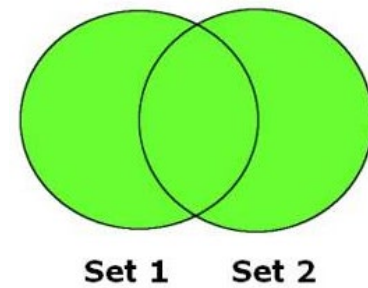
Minus
Except

INTERSECT



- INTERSECT: These are the records that exist in both Dataset1 and Dataset2.
- Example:
 - `select branchNo from Staff intersect select branchno from Branch;`
- MySQL solution:
 - `select branchNo from Staff where branchno in (select branchno from Branch);`
 - `select Staff.branchNo from Staff, Branch where Staff.branchno=Branch.branchno;`
 - `select Staff.branchNo from Staff join on Branch where Staff.branchno=Branch.branchno;`

UNION



- Allows us to **combine** two or more result sets from multiple tables into a single result set.
- There are some rules that we need to follow in order to use the UNION operator:
 1. The **number of columns** that appears in the corresponding SELECT statements **must be equal**.
eg. SELECT **column1, column2** from table1
UNION
SELECT **column1, column2** from table2;
 2. The columns that appear in the corresponding positions of each SELECT statement must have **similar** data types or at least convertible data types.

UNION Syntax

- By default, the UNION operator **eliminates duplicate** rows from the result even if we don't use the DISTINCT operator explicitly.
- To keep the duplicates, use **UNION ALL**
- MySQL Syntax:

```
SELECT column1, column2 from table1
UNION [DISTINCT | ALL]
SELECT column1, column2 from table2
UNION [DISTINCT | ALL]
SELECT column1, column2 from table3
.....
```

Example - Union

- List all cities where there is either a branch office or a property for rent.

```
(select city from Branch )  
UNION (select city from PropertyForRent);
```

city
London
Glasgow
Bristol
London
Aberdeen

UNION

city
Aberdeen
Glasgow
Glasgow
Glasgow
Glasgow
London



city
London
Glasgow
Bristol
Aberdeen

Example – Union All

- This operation is similar to Union. But it also shows the duplicate rows.

```
(select city from Branch )  
UNION ALL (select city from PropertyForRent);
```

city
London
Glasgow
Bristol
London
Aberdeen

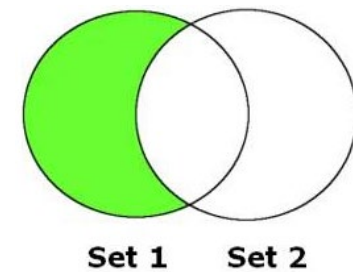
**UNION
ALL**

city
Aberdeen
Glasgow
Glasgow
Glasgow
Glasgow
London



city
London
Glasgow
Bristol
London
Aberdeen
Aberdeen
Glasgow
Glasgow
Glasgow
Glasgow
London

Set Difference



- List all cities where there is a branch office but no properties for rent.

```
select city from Branch where  
city not in (select city from PropertyForRent);
```



Class exercises – IN, NOT IN

- Use Staff and Branch tables in dreamhome
- Show the staff names who work in London or Aberdeen.
- Show the staff names who not work in London and Aberdeen.

Class exercise – Union

- Use EMPLOYEE and CUSTOMERS tables in dreamhome
- List name and address together for all customers and employee
- Concat() function adds two or more expressions together.
 - https://www.w3schools.com/sql/func_mysql_concat.asp

```
SELECT 'Customer', name, concat(Address, ', ', city, ', ', state)  
as Address FROM CUSTOMERS
```

UNION

```
SELECT 'Employee', concat(Fname, ' ', Lname) as name,  
Address FROM EMPLOYEE;
```

Subqueries

- Some SQL statements can have a SELECT embedded within them.
- A **subselect** can be used in WHERE and HAVING clauses of an outer SELECT, where it is called a *subquery* or *nested query*.

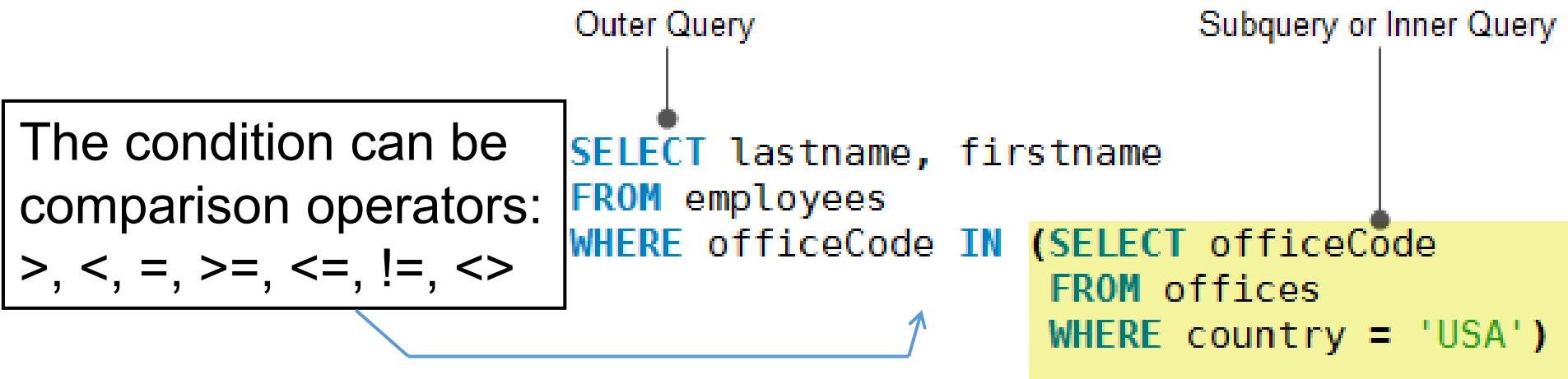
eg: select * from Staff where branchNo in
(select **branchNo** from Staff where position='Manager');

 *subselect/subquery/nested query*

- Subselects may also appear in INSERT, UPDATE, and DELETE statements.
- **Can use subqueries provided result columns come from same table.**

MySQL Subquery

- The following subquery returns employees who locate in the offices in the USA
 - The subquery returns all offices codes of the offices that locate in the USA.
 - The outer query selects the last name and first name of employees whose office code is in the result set returned from the subquery.



Subquery Rules

- ORDER BY clause may not be used in a subquery (although it may be used in outermost SELECT).
eg: `select * from Staff where branchNo in ( select branchNo from Staff where position='Manager') order by fName;`
- Subquery SELECT list must consist of a single column name or expression, except for subqueries that use EXISTS.
- By default, column names refer to table name in FROM clause of subquery. Can refer to a table in FROM using an *alias*.

Class exercises - subqueries

- Question: Find the staff who has the maximum salary.
- Wrong: **select fname, max(salary) from Staff;**

fname	max(salary)
Mary	30000

- Correct: **select fname, salary from Staff
where salary=(select max(salary) from Staff);**

fname	salary
John	30000

Class exercise - subquery

- A subquery is also called an **inner query**, while the query that contains the subquery is called an **outer query**. A subquery returns a result set to the outer query.

List the name of the employees who has female dependents.

Outer
query

```
select Fname, Lname from  
EMPLOYEE as e where ssn IN  
(select essn from DEPENDENT d  
where d.Sex='F');
```

Subquery
(Inner query)

Final output

Fname	Lname
John	Smith
Franklin	Wong

essn
123456789
123456789
333445555
333445555

Example 6.19 Subquery with Equality

- List staff who work in branch at '163 Main Street'.

```
SELECT staffNo, fName,  
        lName, position  
FROM dreamhome.Staff  
WHERE branchNo =  
      (SELECT branchNo  
       FROM dreamhome.Branch  
       WHERE street =  
       '163 Main Street' );
```

staffNo	fName	lName	position
SG14	David	Ford	Supervisor
SG37	Ann	Beech	Assistant
SG5	Susan	Brand	Manager

branchNo	street	city	postcode
B002	56 Cover Drive	London	NW10 6EU
B003	163 Main Street	Glasgow	G11 9QX
B004	32 Manse Road	Bristol	BS99 1NZ
B005	22 Deer Road	London	SW1 4EH
B007	16 Argyll Street	Aberdeen	AB2 3SU

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Example 6.19 Subquery with Equality

- **Inner SELECT** finds branch number for branch at '163 Main St' ('B003').
- Outer SELECT then retrieves details of all staff who work at this branch.

- Outer SELECT then becomes:

**SELECT staffNo, fName,
lName, position
FROM dreamhome.Staff
WHERE branchNo = 'B003';**

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Example 6.20 Subquery with Aggregate

(Q) List all staff whose salary is greater than the average salary, and show how much difference.

- Cannot write 'WHERE salary > AVG(salary)'
- Instead, use subquery to find average salary (17000), and then use outer SELECT to find those staff with salary greater than this:

**SELECT staffNo, fName, lName, position,
salary - 17000 As salDiff
FROM Staff
WHERE salary > 17000;**

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Example 6.20 Subquery with Aggregate #2

<Answer>

```
SELECT staffNo, fName, lName, position,  
salary - (SELECT AVG(salary) FROM Staff) As SalDiff  
FROM Staff  
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

OR

```
SELECT staffNo, fName, lName, position,  
s.salary-avg1.savg as SalDiff  
FROM Staff s, (SELECT AVG(salary) as savg FROM Staff) as avg1  
WHERE s.salary > avg1.savg ;
```

Subqueries with Operands

- Subqueries with operands
 - Syntax: operand **comparison_operator** **ANY** subquery
 - ANY:
 - **must follow a comparison operator,**
 - means “return TRUE if the comparison is TRUE for ANY of the values in the column that the subquery returns.”
 - **comparison_operators** : =, >, <, >=, <=, <>, !=
- eg: **select salary, branchNo from Staff where salary > ANY (select salary from Staff where branchNo='B003');**
- **SOME** is an ISO standard equivalent for **ANY**.
- Returns **TRUE** if the value *v* is equal to **some** value in the set *V* and is hence equivalent to **IN**

<https://dev.mysql.com/doc/refman/8.0/en/any-in-some-subqueries.html>

Class exercises - subqueries with Operands

- Question: list the city and branchno that has staff born before 1980.
- Wrong, runtime error:

```
select city, branchNo from Branch where branchNo =  
    (select branchNo from Staff where DOB < '1980-01-01' );
```

- Correct:

```
select city, branchNo from Branch where branchNo IN  
    (select branchNo from Staff where DOB < '1980-01-01' );
```

Correlated Subquery – IN

- Find the names of employees who are younger than their spouses and have a salary > 60000

```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT Essn FROM DEPENDENT as D
WHERE E.Bdate<D.Bdate AND D.relationship='spouse') and
E.salary>60000;
```

Subquery
result:
(see next page)

Essn
123456789
333445555
444444401
987654321

Final
result:

Fname	Lname
Bonnie	Bays

Using join

```
SELECT E.Fname, E.Lname
FROM DEPENDENT as D,EMPLOYEE AS E
WHERE E.ssn=D.essn and E.bdate < D.bdate and
D.relationship='spouse' and E.salary > 60000;
```

Correlated Subquery – IN

- Find the names of employees who are younger than their spouses and have a salary > 60000

```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT Essn FROM DEPENDENT as D
WHERE E.Bdate<D.Bdate AND D.relationship='spouse') and
E.salary>60000;
```

Subquery (modified) :

```
SELECT Essn FROM DEPENDENT as D, EMPLOYEE E
WHERE D.ESSN=E.SSN and E.Bdate<D.Bdate AND
D.relationship='spouse';
```

Subquery
result:

Essn
123456789
333445555
444444401
987654321

Example 6.21 Nested subquery: use of IN

- List properties handled by staff at '163 Main St'.

```
SELECT propertyNo, street, city, postcode, type,  
rooms, rent
```

```
FROM PropertyForRent
```

```
WHERE staffNo IN
```

```
(SELECT staffNo
```

```
FROM Staff
```

```
WHERE branchNo =
```

```
(SELECT branchNo
```

```
FROM Branch
```

```
WHERE street = '163 Main St'));
```

Example 6.21 Nested subquery: use of IN

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650.00	CO46	SA9	B007
PG16	5 Novar ...	Glasgow	G12 9AX	Flat	4	450.00	CO93	SG14	B003
PG21	18 Dale ...	Glasgow	G12	House	5	600.00	CO87	SG37	B003
PG36	2 Manor ...	Glasgow	G32 4QX	Flat	3	375.00	CO93	SG37	B003
PG4	6 Lawren...	Glasgow	G11 9QX	Flat	3	350.00	CO40	NULL	B003
PL94	6 Argyll ...	London	NW2	Flat	4	400.00	CO87	SL41	B005

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

branchNo	street	city	postcode
B002	56 Cover Drive	London	NW10 6EU
B003	163 Main Street	Glasgow	G11 9QX
B004	32 Manse Road	Bristol	BS99 1NZ
B005	22 Deer Road	London	SW1 4EH
B007	16 Argyll Street	Aberdeen	AB2 3SU

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

Class exercises - subqueries and table scopes

- The inner query can access the tables in the outer query.
- But the outer query cannot access the tables in the inner query, the same concept as variables in the nest blocks.

(Q) Find the female staff first names and their branch cities.

(Answer)

- Wrong, Staff table is not available in the outer query:

```
select fname, city from Branch where branchNo IN  
(select branchNo from Staff where sex = 'F' );
```

- Correct:

```
select fname, city from Staff s, Branch b where  
s.branchNo =b.branchNo and s.branchNo IN  
(select branchNo from Staff where sex = 'F' );
```

Multiple Rows Subqueries

- If a subquery returns more than one row to the outer SQL statement, **the “=” operator is not allowed**.
 - Equal “=” means “identical” – it is one to one comparison if both sides have the same value.
- We must use the **IN, ANY (SOME), or ALL** operator in outer query to handle a subquery that returns multiple rows.
- Syntax: operand **IN** (subquery) , **NOT IN** (subquery)
 - Check whether a value is **within (or NOT within, respectively)** a set of values.

<https://dev.mysql.com/doc/refman/8.0/en/any-in-some-subqueries.html>

Subquery

- Usually needs data from different tables.
 - The join can happen in either a subquery or outer query.
- Do the question in 2 or 3 steps.
 - The first step is usually a subquery.
 - Find the values for the outer query.
- Put the subquery in a pair of ()
 - The () has higher priority to run first.
- Put the () in correct positions

ANY and ALL

- ANY and ALL may be used with subqueries that produce a single column of numbers.
- With ALL, condition will only be **true** if it is satisfied by **all values** produced by subquery.
- With ANY, condition will be **true** if it is satisfied by **any values** produced by subquery.
- If **subquery** is **empty**, **ALL** returns **true**, **ANY** returns **false**.
- SOME may be used in place of ANY.

Example 6.22 Use of ANY/SOME

- Find staff whose salary is larger than salary of **at least one** member of staff at branch B003.

SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > **SOME**
 (**SELECT** salary
 FROM Staff
 WHERE branchNo = 'B003');

staffNo	fName	lName	position	salary
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00
SL21	John	White	Manager	30000.00

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Example 6.22 Use of ANY/SOME

- Inner query produces set {12000, 18000, 24000} and outer query selects those staff whose salaries are greater than any of the values in this set.

SELECT salary
FROM Staff
WHERE
branchNo = 'B003';

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Example 6.23 Use of ALL

- Find staff whose salary is larger than salary of every member of staff at branch B003.

SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > **ALL**
 (**SELECT** salary
 FROM Staff
 WHERE branchNo = 'B003');

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

Multiple Column Subqueries

- The subquery can have more than one column in the output.
- If so, the outer query must have the same number of columns in the condition to match the subquery output list.

```
SELECT ..... FROM ... WHERE (b1, b2) in (  
    SELECT a1, a2 FROM ... WHERE ...)  
);
```

- Exercise on dreamhome database
 - Based on tables:
 - Students, Courses, Students_Courses
 - Find the student name and course name for the students who take the same course more than one time.

Multiple Column Subqueries (Steps)

- Step 1: use Students_Courses table to find the student id and course id for whom took the same course more than one time.
 - Hint: need to use group by and filter
- Step 2: use tables Students and Courses to print the name by matching the student id and course id.
- Result:

student_name	name
Claudia Lee	Java2
Andrew Lin	Java2
Andrew Lin	Data Structure

Multiple Column Subqueries (Answer)

- Step 1:
select sid, cid from Students_Courses group by sid,
cid having count(*) > 1;
- Step 2:
select distinct concat(s.first_name, ' ', s.last_name)
as student_name, c.name
from Students s, Courses c, Students_Courses sc
where s.sid=sc.sid and c.cid=sc.cid
and (s.sid, c.cid) in (
select sid, cid from Students_Courses group by
sid, cid having count(*) > 1
);

EXISTS and NOT EXISTS

- EXISTS and NOT EXISTS are for use only with subqueries.
- Produce a simple true/false result.
- True if and only if there exists at least one row in result table returned by subquery.
- False if subquery returns an empty result table.
- NOT EXISTS is the opposite of EXISTS.
- As (NOT) EXISTS check only for existence or non-existence of rows in subquery result table, subquery can contain any number of columns.

Example 6.31 Query using EXISTS (Semijoin, faster)

- Find all staff who work in a London branch.

SELECT staffNo, fName, lName, position

FROM Staff s

WHERE EXISTS

(SELECT * FROM Branch b

WHERE s.branchNo = b.branchNo **AND** city = 'London');

staffNo	fName	lName	position
SL21	John	White	Manager
SL41	Julie	Lee	Assistant

Example 6.31 Query using EXISTS

- Note, search condition `s.branchNo = b.branchNo` is necessary to consider correct branch record for each member of staff.
- If omitted, would get all staff records listed out because subquery:

```
SELECT * FROM Branch WHERE city='London'
```

- would always be true and query would be:

```
SELECT staffNo, fName, lName, position FROM Staff  
WHERE true;
```

Note: `true` means

`EXISTS`

```
(SELECT * FROM Branch b  
WHERE city = 'London');
```

Example 6.31 Query using EXISTS

```
SELECT staffNo, fName, lName, position  
FROM Staff s  
WHERE EXISTS  
(SELECT * FROM Branch b  
WHERE s.branchNo = b.branchNo AND city = 'London');
```

- Could also write this query using join construct:

```
SELECT staffNo, fName, lName, position  
FROM Staff s, Branch b  
WHERE s.branchNo = b.branchNo  
AND city = 'London';
```


Correlated Nested Queries

- A correlated subquery is a subquery that uses the information from the **outer** query, or we can say that a **correlated subquery depends on the outer query**.
- **Evaluate from inside to outside** and evaluate once for each tuple in the **outer** query
- Example: find the staff that has a salary greater than the average salary of all staff for a particular branchno.

```
SELECT branchNo, fname, salary FROM dreamhome.Staff AS outerTable  
WHERE salary >  
  ( SELECT AVG(salary) FROM dreamhome.Staff  
    WHERE branchNo = outerTable.branchNo );
```

- **The inner query executes for every branchno because the branchno changes for every row.**
- **Hence the average salary will also change.**

MySQL Subquery in FROM Clause

- When we use a subquery in the FROM clause, the **result set** returned from the subquery **is used as a table**. This table is referred to as a ***derived table***.
- Example: find the max #, min #, and avg # of headcounts for 'relationship types' in the DEPENDENT table.

```
SELECT max(count) as max, min(count) as min,  
format(avg(count),1) as avg  
FROM
```

```
(SELECT Relationship, count(Relationship) AS count  
FROM DEPENDENT  
GROUP BY Relationship) AS Dep_info;
```

max	min	avg
5	2	3.7

Multi-Table Queries

- Can use **subqueries** provided result columns come from same table.
- If result columns come from more than one table must use a **join**.
- To perform join, include more than one table in **FROM clause**.
- Use **comma** as separator and typically include **WHERE** clause to specify join column(s).

Multi-Table Queries Alias

- Also possible to use an alias for a table named in FROM clause.
- Alias is separated from table name with a space.
- Alias can be used to qualify column names when there is ambiguity.

Cartesian Product (Joins), CROSS JOIN

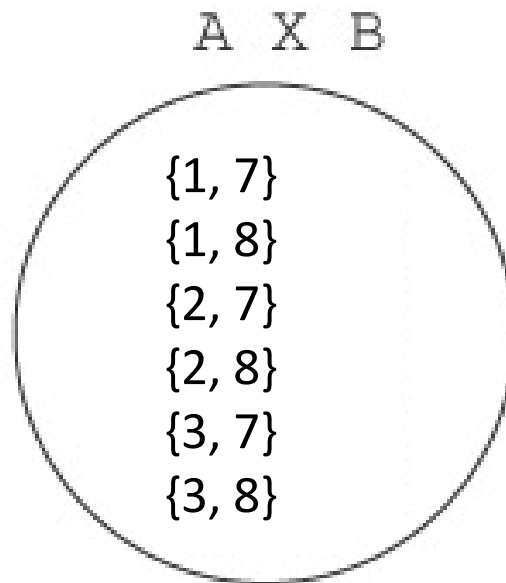
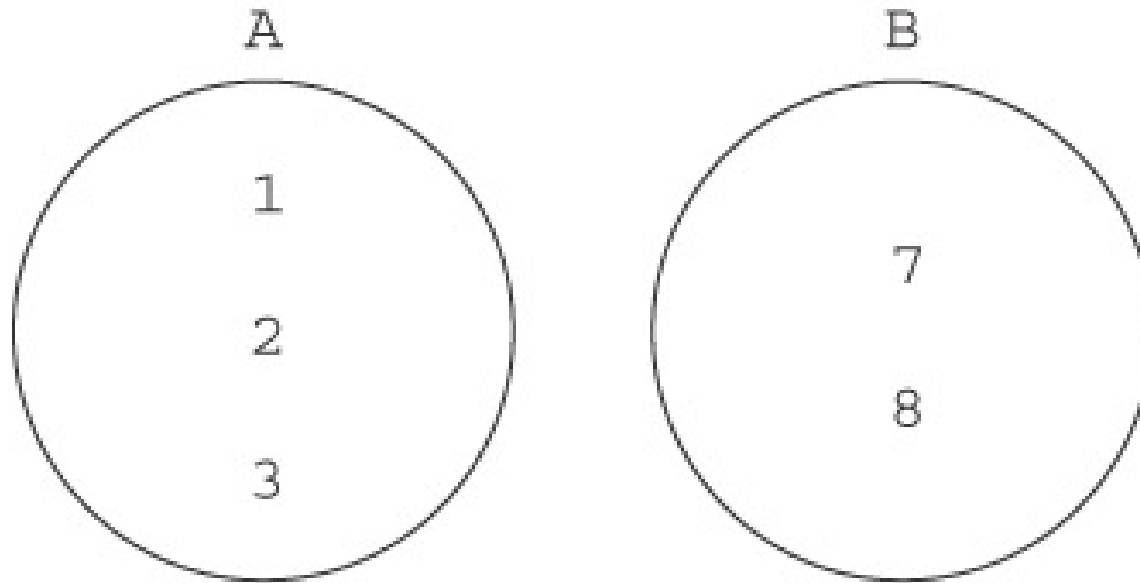
- What happens to the following query?

```
select * from Staff s, Branch b;
```

```
select * from Staff s cross join Branch b;
```

**Always give join condition for
every table listed in the query**

Cartesian product



- The number of combinations
= $N(A) \times N(B)$
- Example: $N(A) = 3$, $N(B) = 2$
 $N(A \times B) = 3 \times 2 \Rightarrow 6$

Example - Cartesian product

- **Select * from Staff;** → 6 rows, 8 cols

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9001.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

- **select * from Branch;** → 5 rows, 4 cols

branchNo	street	city	postcode
B002	56 Cover Drive	London	NW10 6EU
B003	163 Main Street	Glasgow	G11 9QX
B004	32 Manse Road	Bristol	BS99 1NZ
B005	22 Deer Road	London	SW1 4EH
B007	16 Argyll Street	Aberdeen	AB2 3SU

- **select * from Staff, Branch;** → 30 rows (6x5), 12 cols (8+4)

Cartesian Product X (Joins), CROSS JOIN

select fName, sex, city from
Staff, Branch;

select fName, sex, city from
Staff **cross join** Branch;

**Always give join condition for
every table listed in the query**

fName	sex	city
Mary	F	London
Mary	F	Glasgow
Mary	F	Bristol
Mary	F	London
Mary	F	Aberdeen
David	M	London
David	M	Glasgow
David	M	Bristol
David	M	London
David	M	Aberdeen
Ann	F	London
Ann	F	Glasgow
Ann	F	Bristol
Ann	F	London
Ann	F	Aberdeen
Susan	F	London
Susan	F	Glasgow
Susan	F	Bristol
Susan	F	London
Susan	F	Aberdeen
John	M	London
John	M	Glasgow
John	M	Bristol
John	M	London
John	M	Aberdeen
Julie	F	London
Julie	F	Glasgow
Julie	F	Bristol
Julie	F	London
Julie	F	Aberdeen

30 rows in set (0.00 sec)

Computing a Join

- Procedure for generating results of a join are:
 1. Form Cartesian product of the tables named in FROM clause.
 2. If there is a WHERE clause, apply the search condition to each row of the product table, retaining those rows that satisfy the condition.
 3. For each remaining row, determine value of each item in SELECT list to produce a single row in result table.

Computing a Join

4. If DISTINCT has been specified, eliminate any duplicate rows from the result table.
 5. If there is an ORDER BY clause, sort result table as required.
- SQL provides special format of SELECT for Cartesian product:

```
SELECT    [DISTINCT | ALL]      {*} | columnList}
FROM Table1 CROSS JOIN Table2
```

Join and aggregations

- Don't forget the join condition, otherwise, the results will be wrong.
- Give a meaningful alias to each table to make the SQL statement shorter.
- Use the correct column for the aggregations
- Have the where condition to filter individual rows, if the question has such conditions.
- Group the column requested by the question
- Follow the correct sequence:
 - SELECT FROM ... WHERE ... GROUP BY ... HAVING

Case study – join and aggregations

- List the number of staff working in each branch
 - select b.branchno, count(s.staffno) from Staff s, Branch b
where group by b.branchno;
→ wrong, why?
 - select b.branchno, count(s.staffno) from Staff s, Branch b
where b.branchno=s.branchno group by s.staffno;
→ wrong, why?
 - select b.branchno, count(s.staffno) from Staff s, Branch b
where b.branchno=s.branchno group by b.branchno;
→ correct

Class exercises – join, aggregations

- Based on **Staff1** and Branch tables in dreamhome database and list the followings:
 - The number of staff working in each city
 - The number of staff for each gender in each city
 - The number of staff for each gender in each branch

Home exercises - Subquery 1

- Find the staff name who has the minimum salary
- Find the staff name who has the maximum salary for each branch
- Find the staff name who has the maximum salary for each city
- Find the branch that does not have any staff
- Find the city that does not have any staff
- Do exercise paper

Home exercises - Subquery 2

- Related to “minimum salary and staff”
- What do the results tell us?

1. `SELECT fname, salary`

`FROM Staff where salary = (SELECT min(salary) FROM Staff);`

2. `SELECT fname, salary = (SELECT min(salary) FROM Staff)`
`FROM Staff;`

3. `SELECT fname, salary`

`FROM Staff s, (SELECT min(salary) as min_salary FROM Staff) t`
`WHERE s.salary=t.min_salary;`

Home exercise – subquery and aggregations

- Use **Staff1** and Branch tables in dreamhome

(Q) List the staff and salary who has maximum salary in each **branch**

(A) select s.branchno, s.staffno, s.fname, s.lname, s.salary
from **Staff1** s
where (s.branchno, s.salary) in
 (select branchno, max(salary)
 from **Staff1** group by branchno);

→ Cannot use =, why?

(A) Because same branchno might have different staff with the same maximum salary. => subquery return more than 1 row.

Home exercise – join, subquery and aggregations

- Use **Staff1** and Branch tables in dreamhome
- The staff name and salary who has minimum salary in each **city**

```
select b.city, s.fname, s.lname, s.salary from Staff1 s,  
Branch b where s.branchno=b.branchno and (b.city,  
s.salary) in (select city, min(salary) from Staff1 s, Branch  
b where s.branchno=b.branchno group by city);
```

→ Must use group by city

→ Must use city to compare subquery

→ Cannot use = because same branchno might have different staff with the same minimum salary.

Home Exercise

- SQL exercises questions 44 – 57, 74 - 77
- The following SQL & the examples in textbook/slides
 - SELECT
 - AS
 - GROUP BY, HAVING
 - ORDER BY
 - IN, NOT IN
 - BETWEEN AND
 - EXISTS
 - Subquery, IN< ALL, ANY, SOME
 - **Aggregates functions:** MAX, COUNT, MIN, AVG, SUM
 - INNER JOIN, OUTER JOIN

Leetcode.com SQL exercise

- 183. Customers Who Never Order
 - <https://leetcode.com/problems/customers-who-never-order/>
- Suppose that a website contains two tables, the Customers table and the Orders table. Write a SQL query to find all customers who never order anything.

Table: Customers.

Id	Name
1	Joe
2	Henry
3	Sam
4	Max

Table: Orders.

Id	CustomerId
1	3
2	1

Customers
Henry
Max

Output

Leetcode.com SQL exercise

- 184. Department Highest Salary
 - <https://leetcode.com/problems/department-highest-salary/>
- Write a SQL query to find employees who have the highest salary in each of the departments. For the above tables, your SQL query should return the following rows (order of rows does not matter).

Id	Name	Salary	DepartmentId
1	Joe	70000	1
2	Jim	90000	1
3	Henry	80000	2
4	Sam	60000	2
5	Max	90000	1

Employee table

Id	Name
1	IT
2	Sales

Department table

Department	Employee	Salary
IT	Max	90000
IT	Jim	90000
Sales	Henry	80000

Output

Leetcode.com SQL exercise

- 197. Rising Temperature
 - <https://leetcode.com/problems/rising-temperature/>
- Given a Weather table, write a SQL query to find all dates' Ids with higher temperature compared to its previous (yesterday's) dates.

Id(INT)	RecordDate DATE	Temperature(INT)
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

Id
2
4

Output

Conclusion

- Understand set
- Practice on subquery
- Do a lot of SQL query exercise
- Work on homework and project
- Seek help from Samurai as early as possible